Data Mining and Machine Learning

Gennaro Basile

March 23, 2026

Contents

1 Sommario **3**

2 Introduction to Artificial Neural Networks **5**
2.1 What is a Perceptron? 5
2.1.1 Recap: from Perceptrons to ANNs 5

3 The Birth of Artificial Neural Networks (ANNs) **5**
3.1 1. Non-Linearity (The Death of the Step Function) 6
3.2 2. Backpropagation 6
3.3 3. Computational Power 6
3.4 From Perceptrons to ANNs 6

4 Multi-Layer Perceptron (MLP) Architecture **6**
4.1 Example: MNIST Digit Classification 7

5 The Mathematics of Forward Propagation **7**

6 Universal Approximation Theorem **7**

7 The Power of Hidden Layers: Feature Hierarchies **7**

8 Introduction to Artificial Neural Networks (ANNs) **8**
8.1 Overview and Recap 8
8.2 Architecture of a Multi-Layer Perceptron 8
8.2.1 A Simple Layout 8
8.2.2 Template Case: Recognizing Handwritten Digits (MNIST) 8
8.2.3 Algebraic Description of a Fully Connected Network 9
8.3 Universal Approximation Theorems 10
8.4 The Role of Hidden Layers: Abstraction and Feature Hierarchies 11
8.4.1 Example: Decomposing Handwritten Digits 11
8.4.2 A Visual Interpretation 11

1 Sommario

- **Introduzione e Recap** (Slides 1–3)
  - Titolo e overview del corso (Slides 1–2)
  - Recap: limiti dei perceptron, necessità di non-linearità, backpropagation e potenza computazionale (Slide 3)
- **Architettura di un MLP** (Slides 4–14)
  - Layout di base: input layer, hidden layer, output layer, feedforward (Slide 4)
  - Caso template: riconoscimento cifre scritte (MNIST), knowledge base, training/test set (Slides 5–6)
  - Descrizione algebrica: vettori di attivazione, matrici dei pesi, preattivazione, funzione di attivazione (Slides 7–9)
  - Universal Approximation Theorems: width vs. depth (Slide 10)
  - Ruolo degli hidden layers: livelli di astrazione, gerarchia di features (Slides 11–14)
- **Dissecting a Simple MLP — Il caso XOR** (Slides 15–18)
  - Struttura della rete XOR: input, pesi W_1 e W_2, hidden layer, output layer (Slides 15–16)
  - Funzione di attivazione sigmoid e funzione SoftMax (Slide 18)
- **The Forward Step** (Slides 19–23)
  - Inizializzazione dei pesi (Slide 20)
  - Calcolo di $\mathbf{z}_1 = XW_1$ e applicazione della sigmoid: $\mathbf{h}_1 = \sigma(\mathbf{z}_1)$ (Slide 21)
  - Calcolo del secondo layer: $\mathbf{z}_2 = \mathbf{h}_1 W_2$ (Slide 22)
  - Applicazione della SoftMax all'output: $\mathbf{h}_2 = \text{SoftMax}(\mathbf{z}_2)$ (Slide 23)
- **Backpropagation — The Backward Step** (Slides 24–35)
  - Obiettivo: aggiornamento pesi per minimizzare la loss function (Slide 25)
  - Loss function con regolarizzazione Ridge ($\mathcal{L}^2$) (Slide 26)
  - Prima backpropagation: $d\mathcal{L}/dW_2$ tramite chain rule (Slides 27–30)
  - Calcolo numerico delle tre derivate parziali e aggiornamento di W_2 (Slides 29–30)
  - Seconda backpropagation: $d\mathcal{L}/dW_1$ con riutilizzo dei gradienti (Slides 31–35)
- **Applicazione: XOR Operator con MLP** (Slides 36–38)
  - Costruzione dell'operatore XOR, non linearità necessaria (Slide 37)
  - Riferimento al notebook GitHub (Slide 38)
- **Terminologia e Concetti Generali sugli MLP** (Slides 39–42)
  - Il Multi Layer Perceptron come ANN più semplice (Slide 39)
  - Terminologia: neuroni, layers, pesi (Slides 40–41)
  - Knowledge base: training set, test set, validation set (Slide 42)
- **Supervised Learning Workflow** (Slides 43–50)
  - Forward pass, cost function, firing dei neuroni (Slide 43)
  - Schema classificazione supervisionata: Phase 1 (training) e Phase 2 (inference) (Slide 46)
  - Apprendimento per esempi: cost function, ottimizzazione, backpropagation (Slide 47)
  - Esempio MNIST: normalizzazione, inizializzazione, training iterativo (Slides 48–50)
- **Prospettive e Aspetti Avanzati** (Slides 51–56)
  - Argomenti futuri: architettura, normalizzazione, activation functions, cost function, ottimizzatori (GD, SGD, RMSProp, ADAM), underfitting, overfitting, vanishing gradients (Slide 51)
  - Connessioni AI–neuroscienze, riferimento a *A Thousand Brains* (Slide 53)

– Complessità computazionale e regole pratiche (Slides 54–56)

2 Introduction to Artificial Neural Networks

The lecture introduces the Multi-Layer Perceptron (MLP), covering its basic architecture, the algebraic description of feed-forward networks, the Universal Approximation Theorems, and the role of hidden layers in learning hierarchical representations.

2.1 What is a Perceptron?

To understand Artificial Neural Networks (ANNs), you first need to know what they evolved from: the **Perceptron**.

Invented in the late 1950s, a Perceptron is the simplest form of an artificial neuron. It works in three steps:

1. It takes multiple inputs and multiplies each by a specific **weight**.
2. It sums these weighted inputs together and adds a **bias** term.
3. It passes this sum through a **Step Function** (a hard threshold). If the sum is above a certain number, it outputs a 1. If it's below, it outputs a 0.

While a single perceptron is great at acting like a simple logic gate (like an AND or OR gate), it is severely limited. For complex problems, humans had to *manually* figure out the exact weights and biases to make the perceptron output the right answer. It could not learn complex patterns on its own.

2.1.1 Recap: from Perceptrons to ANNs

In previous lectures we saw that a single **perceptron** can implement simple logic gates. However, using perceptrons directly for complex computations is cumbersome: every new task requires manual, ad-hoc tuning of the weights and biases.

The key insight is that we can build **learning algorithms**, i.e. Artificial Neural Networks (ANNs), that automatically adjust the weights and biases of a network of artificial neurons. This adjustment happens entirely in response to external data, with no direct human intervention at the level of individual weights.

Three ingredients were necessary before real progress could be made:

1. **Non-linearity.** For a network to *learn*, a small change in a weight must produce a correspondingly small change in the output. Step functions (as used in the original perceptron) do not satisfy this requirement, because their output jumps abruptly. Smooth, differentiable activation functions (such as the sigmoid) solve this problem.
2. **Backpropagation.** An efficient algorithm that computes the gradient of the loss function with respect to every weight in the network, enabling systematic weight updates.
3. **Computational power.** Training large networks requires performing billions of floating-point operations, which only became feasible with modern hardware (GPUs, TPUs).

3 The Birth of Artificial Neural Networks (ANNs)

The fundamental breakthrough of ANNs was shifting from *manual tuning* to *automatic learning*. Instead of humans calculating weights, we let an algorithm adjust the weights of thousands of interconnected neurons automatically, based entirely on the data it sees.

However, to make this jump from a rigid Perceptron to a learning, dynamic ANN, scientists needed three critical ingredients:

3.1 1. Non-Linearity (The Death of the Step Function)

Learning in an algorithm means figuring out the "slope" or "gradient" of an error to know which way to adjust the weights.

- **The Perceptron Problem:** The Step Function is flat everywhere except at its jump, meaning its mathematical derivative is almost always zero. If the derivative is zero, the network receives no feedback on how to improve. A small tweak to a weight either does absolutely nothing, or violently flips the output from 0 to 1.
- **The ANN Solution:** We replaced the hard Step Function with smooth, differentiable **Activation Functions** (like the Sigmoid or ReLU). Because they are smooth curves, a tiny adjustment to a weight results in a tiny, predictable change in the output. This allows the network to gradually "slide" down the error curve toward the correct answer.

3.2 2. Backpropagation

Once we had smooth activation functions, we needed a way to apply those gradients across multiple layers of neurons. **Backpropagation** is simply the algorithm that calculates exactly who is to blame for an error.

When the network makes a wrong prediction, backpropagation calculates the gradient of the loss (the error) and works backward from the output layer to the input layer. It mathematically dictates exactly how much every single weight and bias in the entire network needs to be adjusted to make a better prediction next time.

3.3 3. Computational Power

The math behind backpropagation and forward passes isn't conceptually difficult, but the sheer volume is staggering. Updating a deep neural network requires billions of matrix multiplications (floating-point operations). The theories for ANNs existed in the 1980s, but it wasn't until modern hardware—specifically **GPUs (Graphics Processing Units)** and **TPUs (Tensor Processing Units)**—became available that we had the raw computational muscle to actually train them in a reasonable timeframe.

3.4 From Perceptrons to ANNs

Single perceptrons require manual weight tuning, making them impractical for complex tasks. Artificial Neural Networks (ANNs) solve this by automatically learning from data. This evolution required three key breakthroughs:

1. **Non-linearity:** Smooth, differentiable activation functions (e.g., Sigmoid, ReLU) allow small weight adjustments to predictably change outputs. This is a strict requirement for gradient-based learning.
2. **Backpropagation:** An efficient algorithm to compute gradients and update weights systematically across all layers.
3. **Computational Power:** Modern hardware (GPUs, TPUs) to handle massive parallel matrix operations.

4 Multi-Layer Perceptron (MLP) Architecture

A Feed-Forward Neural Network (or MLP) processes information in one direction: **Input Layer** → **Hidden Layer(s)** → **Output Layer**. Neurons in a layer do not connect to each other; they only feed their output forward to *all* neurons in the subsequent layer.

4.1 Example: MNIST Digit Classification

- **Input:** 28×28 grayscale images, flattened into a 784-dimensional vector. While 2D spatial layout is implicitly lost, the patterns remain encoded in the vector indices.
- **Task:** Supervised learning to classify images into one of 10 digits (0-9).
- **Data Split:** The knowledge base is divided into a **Training set** (to adjust weights) and a **Testing set** (to evaluate generalization), where the training set is significantly larger.

5 The Mathematics of Forward Propagation

The core of a neural network is a repeated sequence of linear transformations followed by non-linear activations.

Let $\mathbf{a}^{(h)}$ be the activation vector of layer h, $\mathbf{W}^{(h)}$ the weight matrix connecting layer h to $h+1$, and $\mathbf{w}_0^{(h)}$ the bias vector. For the next layer $h+1$, we compute the preactivation and apply the element-wise activation function σ:

$$\mathbf{a}^{(h+1)} = \sigma\left(\mathbf{W}^{(h)}\mathbf{a}^{(h)} + \mathbf{w}_0^{(h)}\right) \tag{1}$$

An entire network is simply a nested composition of these layer-wise operations:

$$\mathbf{O} = \sigma_L\left(\ldots\sigma_1\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{w}_0^{(1)}\right)\ldots\right) \tag{2}$$

Takeaway: Every layer performs a matrix multiplication followed by an element-wise non-linear function.

6 Universal Approximation Theorem

This theorem states that a feed-forward network with a **single hidden layer** and a non-linear activation function can approximate *any* continuous function, provided it has enough neurons (sufficient width). A similar result holds for deeper networks (sufficient depth).

Crucial Data Science Caveat: This is an *existence* theorem. It guarantees a mathematical solution exists, but it tells us nothing about *how* to find the optimal weights, how many neurons are actually needed, or if gradient descent will successfully converge to that solution.

7 The Power of Hidden Layers: Feature Hierarchies

Deep networks excel because hidden layers learn **hierarchical representations** of data.

- **Early layers:** Learn basic, low-level features (e.g., edges, pixels).
- **Middle layers:** Combine basic features into intermediate shapes (e.g., loops, curves).
- **Output layers:** Combine shapes to recognize complex concepts (e.g., the digit "8").

This layered decomposition allows networks to reuse fundamental primitive patterns across different classes, making them incredibly powerful for complex unstructured data like images, text, and audio.

8 Introduction to Artificial Neural Networks (ANNs)

These notes cover Lecture 4.1 of the Machine Learning course, delivered on March 12, 2026.

8.1 Overview and Recap

The lecture is organized around four main topics: (1) dissecting a simple MLP, (2) the forward step, (3) backpropagation (the backward step), and (4) using an MLP as a XOR operator. These notes focus on the first part: understanding the architecture of a simple MLP (slides 1–14).

8.2 Architecture of a Multi-Layer Perceptron

8.2.1 A Simple Layout

Feed-Forward Neural Network

A **feed-forward neural network** (also called a Multi-Layer Perceptron, MLP) is a network of artificial neurons organized in **layers**, where information flows in one direction only: from the **input layer**, through one or more **hidden layers**, to the **output layer**. Neurons within the same layer are *not* connected to each other; each neuron connects only to neurons in the immediately subsequent layer.

In a diagram each circle represents a neuron and each line represents a weighted connection. The key properties of this layout are:

- **Input layer:** receives the raw data (e.g. pixel values of an image). Each neuron holds one component of the input vector.
- **Hidden layer(s):** process the information coming from the previous layer. Each successive hidden layer builds higher-level, more abstract representations of the data.
- **Output layer:** produces the final prediction (e.g. a class label).

An important detail: the output of each neuron is *unique* — that is, a neuron produces a single scalar value, and that same value is sent to *all* neurons in the next layer.

8.2.2 Template Case: Recognizing Handwritten Digits (MNIST)

To make the architecture concrete, consider the classic problem of recognizing handwritten digits from the MNIST dataset.

MNIST Classification Problem

Given a grayscale image of a handwritten digit (size 28×28 pixels), classify it into one of 10 classes: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

The setup is as follows:

- Each image is a 28×28 matrix of pixel intensities. For the network, this matrix is **flattened** into a single vector of length $28 \times 28 = 784$. The spatial structure of the image is lost in this flattening, but the information is still present in the 784-dimensional vector.
- We have N such images, each paired with its correct label (this is **supervised learning**). This collection of image–label pairs is called the **knowledge base** (KB).
- The knowledge base is split into two subsets:
  - N' images for **training** (used to adjust weights), and
  - N'' images for **testing** (used to evaluate performance),

with $N' + N'' = N$ and typically $N' \gg N''$.

The flattening process (from a 28×28 grid to a 784-element vector) is simply a rearrangement. Although the two-dimensional spatial layout is no longer explicit, the network can still discover the relevant patterns because the pixel values and their relative positions (encoded by their index in the vector) remain unchanged.

8.2.3 Algebraic Description of a Fully Connected Network

We now describe the mathematics that governs how data flows through the network.

Notation for Layers, Activations, and Weights

- Let L^h, $h = 1, \ldots, M$, denote the hidden layers of the network.
- The **activation vector** of layer L^h is the column vector:

$$\mathbf{a}^h = (a_1^h,\ a_2^h,\ \ldots,\ a_{p_h}^h)^\top,$$

  where p_h is the number of neurons in layer h.
- The **weight matrix** connecting layer L^h to layer L^{h+1} is denoted $\mathbf{W}^{(h)}$. It is a $p_{h+1} \times p_h$ matrix whose element $w_{ij}^{(h)}$ is the weight from neuron j in layer h to neuron i in layer $h+1$.
- The **bias vector** for layer $h+1$ is $\mathbf{w}_0^h$.

Preactivation and Activation. For each neuron i in layer $h+1$, we first compute the **preactivation** (also called the *weighted input*):

$$z_i^{(h+1)} = \sum_{j=0}^{p_h} w_{ij}^{(h)}\, a_j^{(h)},$$

where the sum from $j = 0$ includes the bias term (with $a_0^{(h)} = 1$ by convention).

Then, a non-linear, differentiable function σ — the **activation function** — is applied element-wise to obtain the activation of each neuron:

$$a_i^{(h+1)} \equiv \sigma\big(z_i^{(h+1)}\big).$$

Matrix form. Writing this for all neurons simultaneously, we get the compact matrix equation:

Fundamental Equation of a Feed-Forward Layer

$$\mathbf{a}^{(h+1)} = \sigma\big(\mathbf{W}^{(h)} \cdot \mathbf{a}^{(h)} + \mathbf{w}_0^h\big),$$

where σ is applied **element-wise** to the resulting vector.

In full matrix notation (absorbing the bias into the weight matrix by appending a column and adding a constant entry $a_0^{(h)} = 1$ to the activation vector):

$$\sigma\left(\begin{bmatrix} a_1^{(h)} \\ \vdots \\ a_{p_h}^{(h)} \\ a_0^{(h)} \end{bmatrix} \times \begin{bmatrix} w_{11}^{(h)} & w_{12}^{(h)} & \cdots & w_{1,p_h}^{(h)} & w_0^{(h)} \\ w_{21}^{(h)} & w_{22}^{(h)} & \cdots & w_{2,p_h}^{(h)} & w_0^{(h)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ w_{p_{h+1},1}^{(h)} & w_{p_{h+1},2}^{(h)} & \cdots & w_{p_{h+1},p_h}^{(h)} & w_0^{(h)} \end{bmatrix}\right) = \begin{bmatrix} a_1^{(h+1)} \\ \vdots \\ a_{p_{h+1}}^{(h+1)} \end{bmatrix}.$$

This single equation is, in essence, the *entire mathematics* of neural networks. Every layer performs the same operation: a matrix multiplication followed by an element-wise non-linear function.

Recursive description of the full network. Consider an MLP with 1 input layer, $L-1$ hidden layers, and 1 output layer. Let $\mathbf{x} \in \mathbb{R}^n$ be the input vector and $\mathbf{o} \in \mathbb{R}^m$ the output vector. Define the pre-activation vector $\mathbf{z}^{h+1} \equiv \mathbf{W}^h \mathbf{a}^h + \mathbf{w}_0^h$. Then the network operates as:

$$\mathbf{a}^1 = \sigma_1(\mathbf{z}^{(0)}), \tag{3}$$

$$\mathbf{a}^{h+1} = \sigma_{h+1}(\mathbf{z}^h), \quad \forall\, h \in \{1, \dots, L-1\}, \tag{4}$$

$$\mathbf{O} = \sigma_L(\mathbf{z}^{L-1}). \tag{5}$$

In principle, each layer can use a different activation function σ_h. In practice, hidden layers usually share the same activation function ($\sigma_1 = \sigma_2 = \cdots = \sigma_{L-1}$, e.g. ReLU or sigmoid), while the output layer uses a different one (σ_L, e.g. softmax for classification).

Putting it all together, the entire network can be written as a composition of functions:

Neural Network as Function Composition

$$\mathbf{O} = \sigma_L\Big(\sigma_{L-1}(\cdots \sigma_1(\mathbf{W}^{(1)}\,\mathbf{x}) \cdots)\Big).$$

A neural network is nothing more than a succession of activation functions applied point-wise to vectors that are produced by successive matrix multiplications.

8.3 Universal Approximation Theorems

A natural question arises: can a neural network actually learn any function we want? The **Universal Approximation Theorems** provide a positive (but limited) answer.

Universal Approximation Theorem (Hornik et al., 1988)

A feed-forward neural network with a single hidden layer and a non-polynomial activation function (e.g. sigmoid, ReLU) can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to any desired degree of accuracy, provided the hidden layer has sufficiently many neurons.

There are two flavors of this result:

1. **Width approach:** keep the network shallow (one hidden layer) but make it *wider* by increasing the number of neurons in that layer.
2. **Depth approach:** keep the width fixed but make the network *deeper* by adding more hidden layers.

Both strategies can achieve universal approximation in theory.

Important caveat. These are **existence theorems**. They guarantee that a network with the right structure *exists*, but they say nothing about:

- how many neurons/layers are actually needed,
- how to find the correct values of the weights (i.e. they do not provide a training algorithm),
- whether gradient-based optimization will converge to the optimal solution.

In practice, finding good weights is the hard part — and that is the job of training algorithms like backpropagation combined with gradient descent.

8.4 The Role of Hidden Layers: Abstraction and Feature Hierarchies

Why do we need hidden layers at all? The answer lies in the concept of **hierarchical abstraction**.

Hierarchical Feature Learning

Hidden layers allow the network to learn a **hierarchy of representations** from the data. Layers closer to the input learn simple, low-level features (e.g. edges, basic shapes), while layers closer to the output learn complex, high-level features (e.g. object parts, entire objects).

This principle applies across different data modalities:

- **Images:** an image is composed of objects, which are composed of parts, which are composed of edges and textures. Early layers detect edges; middle layers detect shapes (e.g. loops, strokes); later layers detect complete objects (e.g. digits).
- **Text:** a sentence is composed of words, which are composed of syllables, which are composed of characters. Each level follows specific patterns that the network can learn.
- **Sound:** audio is composed of musical phrases, which are composed of notes organized at different levels.

8.4.1 Example: Decomposing Handwritten Digits

Consider the MNIST digit recognition task. Any handwritten digit can be decomposed into a set of elementary shapes (strokes, loops, line segments). For example:

- A "9" can be decomposed into a loop (top) and a vertical stroke (bottom).
- An "8" can be decomposed into two loops.
- A "4" can be decomposed into a vertical stroke, a shorter vertical stroke, and a horizontal bar.

Crucially, some elementary patterns are shared across different digit classes. The loop that appears in a "9" is similar to the loop in an "8" or a "0". The network exploits this by having hidden neurons that detect these shared primitives, and later layers combine them to distinguish between different digits.

In terms of neuron activations, similar shapes in the input produce similar firing patterns in the hidden layers. This is precisely what we mean by the network learning *internal representations*.

8.4.2 A Visual Interpretation

In the MNIST network (784 input neurons, several hidden layers, 10 output neurons), the last hidden layer before the output can be thought of as containing neurons that each represent a specific elementary shape (e.g. a loop, a stroke, a curve). The output neurons then combine these shape detectors to decide which digit is present.

This layered decomposition — from raw pixels to edges to shapes to digits — is the fundamental reason why deep networks are so powerful. Each layer performs a relatively simple transformation, but the composition of many such transformations can capture extremely complex patterns.